

AGILE LAB FAT EXAM

Deepak C 23MIS0374

Payment Processing System

Problem Statement: Ensure food orders are placed and processed correctly without errors

Tasks:

1. Develop validation logic
 2. Write JUnit tests for transaction success/failure
 3. Implement CI/CD pipeline with automated testing
 - > Set up version control using GitHub
 - > Configure CI pipeline (e.g., Jenkins/GitHub Actions) to trigger on code commits Automate build process using Maven
 - > Integrate automated unit tests in pipeline
 - Configure CD pipeline for automatic deployment to staging/production
 4. Deploy services on Docker and Kubernetes
 - > Containerize application using Docker (write Dockerfile)
 - > Build and push Docker image to container registry (Docker Hub)
- Create Kubernetes deployment and service YAML files

OrderProcessor.java

```
import java.util.*;
```

```
public class OrderProcessor {
```

```
    public static String placeOrder(Map<String, Object> order) {
```

```

    if (order == null) {
        return "FAILED: Order is null";
    }

    if (!order.containsKey("itemName") || order.get("itemName")
== null) {
        return "FAILED: Item name is missing";
    }

    if (!order.containsKey("quantity") || (int)
order.get("quantity") <= 0) {
        return "FAILED: Invalid quantity";
    }

    if (!order.containsKey("price") || (double) order.get("price")
<= 0.0) {
        return "FAILED: Invalid price";
    }

    return "SUCCESS: Order placed for " +
order.get("itemName");
}

```

```

public static double calculateTotal(int quantity, double price) {
    if (quantity <= 0 || price <= 0.0) {
        return -1.0;
    }

    return quantity * price;
}

```

```

public static void main(String[] args) {
    Map<String, Object> order = new HashMap<>();
    order.put("itemName", "Laptop");
    order.put("quantity", 2);
    order.put("price", 50000.0);

    System.out.println(placeOrder(order));
    System.out.println("Total: " + calculateTotal(2, 50000.0));
}
}

```

OrderProcessorTest.java

```

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import java.util.*;

```

```

public class OrderProcessorTest {

```

```

    // — SUCCESS TESTS —

```

```

    @Test

```

```

    void testOrderPlacedSuccessfully() {
        Map<String, Object> order = new HashMap<>();
        order.put("itemName", "Laptop");
    }
}

```

```
    order.put("quantity", 2);
    order.put("price", 50000.0);
    assertEquals("SUCCESS: Order placed for Laptop",
        OrderProcessor.placeOrder(order));
}
```

```
@Test
void testCalculateTotalCorrect() {
    assertEquals(100000.0, OrderProcessor.calculateTotal(2,
50000.0));
}
```

```
@Test
void testSingleItemOrder() {
    Map<String, Object> order = new HashMap<>();
    order.put("itemName", "Phone");
    order.put("quantity", 1);
    order.put("price", 20000.0);
    assertTrue(OrderProcessor.placeOrder(order).startsWith("SUC
CESS"));
}
```

```
// — FAILURE TESTS —
```

@Test

```
void testNullOrderFails() {  
    assertEquals("FAILED: Order is null",  
        OrderProcessor.placeOrder(null));  
}
```

@Test

```
void testMissingItemNameFails() {  
    Map<String, Object> order = new HashMap<>();  
    order.put("quantity", 2);  
    order.put("price", 500.0);  
    assertTrue(OrderProcessor.placeOrder(order).startsWith("FAL  
LED"));  
}
```

@Test

```
void testZeroQuantityFails() {  
    Map<String, Object> order = new HashMap<>();  
    order.put("itemName", "Tablet");  
    order.put("quantity", 0);  
    order.put("price", 15000.0);  
    assertTrue(OrderProcessor.placeOrder(order).startsWith("FAL  
LED"));  
}
```

@Test

```
void testNegativePriceFails() {  
    Map<String, Object> order = new HashMap<>();  
    order.put("itemName", "Watch");  
    order.put("quantity", 1);  
    order.put("price", -100.0);  
    assertTrue(OrderProcessor.placeOrder(order).startsWith("F A I  
L E D"));  
}
```

@Test

```
void testInvalidTotalReturnsNegative() {  
    assertEquals(-1.0, OrderProcessor.calculateTotal(0, 500.0));  
}  
}
```

The screenshot shows an IDE with the following components:

- Explorer:** A file tree on the left showing the project structure: `Agile Fat` (containing `src`, `target`, `deployment.yaml`, `Dockerfile`, `mvn_output.txt`, and `pom.xml`), `OrderProcessor.java`, `test.java`, and `OrderProcessorTest.java`.
- Dockerfile:** The active editor shows a Dockerfile with the following content:

```
1 FROM eclipse-temurin:17-jdk-alpine
2 WORKDIR /app
3 COPY target/*.jar app.jar
4 EXPOSE 8080
5 ENTRYPOINT ["java", "-jar", "app.jar"]
```
- Terminal:** The bottom panel shows the output of a Maven test run. The command executed is `mvn clean package`. The output includes:

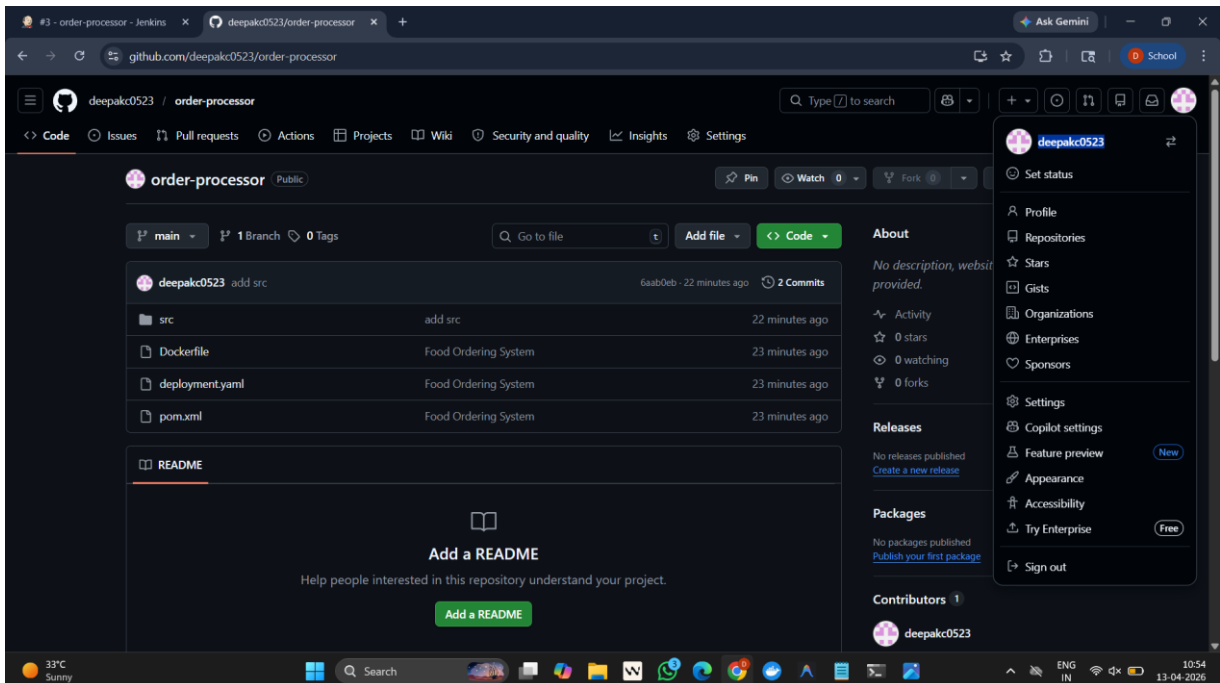
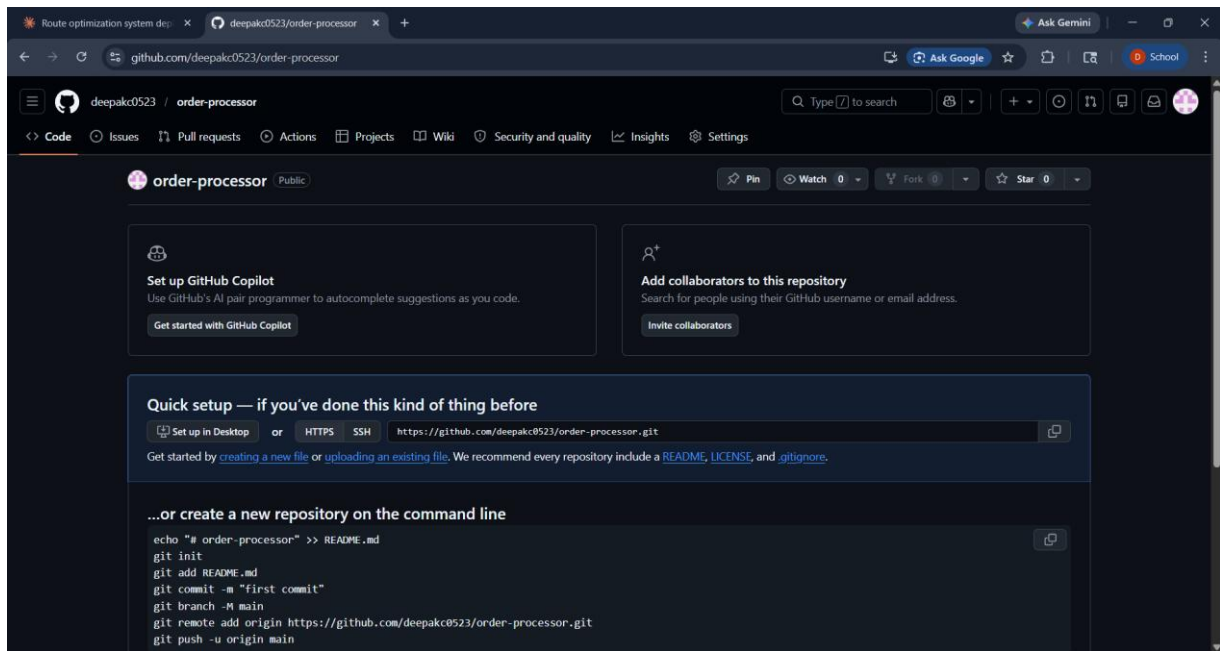
```
PS C:\Users\deepa\OneDrive\Desktop\Agile Fat> mvn clean package
[INFO] --- surefire:3.2.5:test (default-test) @ order-processor ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running OrderProcessorTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.159 s -- in OrderProcessorTest
[INFO] Results:
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jar:3.4.1:jar (default-jar) @ order-processor ---
[INFO] Building jar: C:\Users\deepa\OneDrive\Desktop\Agile Fat\target\order-processor-1.0.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.526 s
[INFO] Finished at: 2026-04-13T10:26:13+05:30
[INFO] -----
PS C:\Users\deepa\OneDrive\Desktop\Agile Fat>
```

The screenshot shows an IDE with the following components:

- Explorer:** A file tree on the left showing the project structure: `Agile Fat` (containing `src`, `target`, `deployment.yaml`, `Dockerfile`, `mvn_output.txt`, and `pom.xml`), `OrderProcessor.java`, `test.java`, and `OrderProcessorTest.java`.
- Dockerfile:** The active editor shows a Dockerfile with the following content:

```
1 FROM eclipse-temurin:17-jdk-alpine
2 WORKDIR /app
3 COPY target/*.jar app.jar
4 EXPOSE 8080
5 ENTRYPOINT ["java", "-jar", "app.jar"]
```
- Terminal:** The bottom panel shows the output of a Maven test run. The command executed is `mvn clean test`. The output includes:

```
PS C:\Users\deepa\OneDrive\Desktop\Agile Fat> mvn clean test
[INFO] --- surefire:3.2.5:test (default-test) @ order-processor ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running OrderProcessorTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.190 s -- in OrderProcessorTest
[INFO] Results:
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 9.029 s
[INFO] Finished at: 2026-04-13T10:24:54+05:30
[INFO] -----
PS C:\Users\deepa\OneDrive\Desktop\Agile Fat>
```



Jenkins / order-processor / #2 / Stages

Started by Deepak Started 13 sec ago Queued 11 ms Build has been executing for 13 sec

Graph

Start Tool Install Clone Repository Build & Run Tests Package Application Build Docker Image Push to DockerHub Deploy to Kubernetes End

Package Application 7s Started 7s ago Jenkins

- Use a tool from a predefined Tool Installation `maven` 95ms
- Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step. 0.13s
- Windows Batch Script `mvn package -DskipTests` 6s

1 C:\ProgramData\Jenkins\jenkins\workspace\order-processor>mvn package -DskipTests

Jenkins / order-processor / #3 / Stages

Started by Deepak Started 8 sec ago Queued 6 ms Build has been executing for 8 sec

Graph

Start Tool Install Clone Repository Build & Run Tests Package Application Build Docker Image Push to DockerHub Deploy to Kubernetes End

Deploy to Kubernetes 1s Started 14m ago Jenkins

- Use a tool from a predefined Tool Installation `maven` 81ms
- Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step. 0.1s
- Windows Batch Script `kubectl apply -f deployment.yaml --validate=false --kubeconfig=C:\Users\deepa\.kube\config` 1s

1 C:\ProgramData\Jenkins\jenkins\workspace\order-processor>kubectl apply -f deployment.yaml --validate=false --kubeconfig=C:\Users\deepa\.kube\config

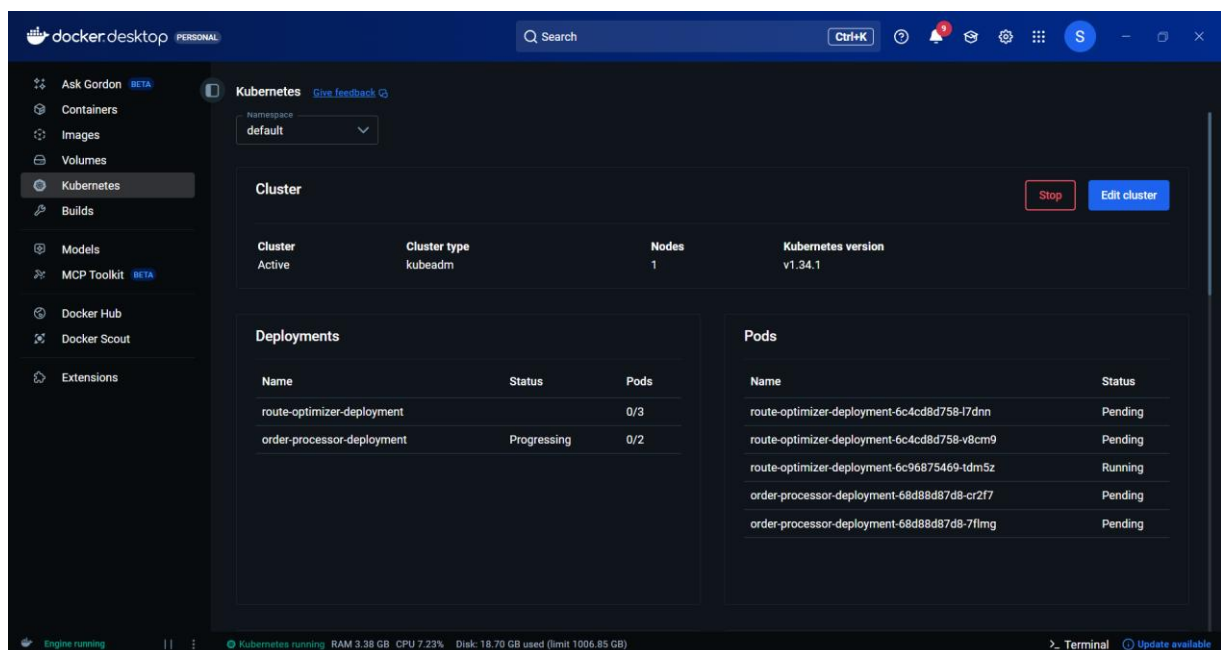
2 deployment.apps/order-processor-deployment created

3 service/order-processor-service created

```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\deepa>kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes           ClusterIP   10.96.0.1     <none>         443/TCP          7d11h
order-processor-service NodePort    10.103.86.39  <none>         80:30008/TCP     3m35s
route-optimizer-service NodePort    10.102.80.251 <none>         80:30007/TCP     15h

C:\Users\deepa>
```



The pipeline Script:

```
pipeline {
    agent any
    tools {
        maven 'maven'
```

```
}  
environment {  
    DOCKER_IMAGE = "sauldee/order-processor:latest"  
}  
stages {  
    stage('Clone Repository') {  
        steps {  
            git branch: 'main',  
            url: 'https://github.com/deepakc0523/order-processor.git'  
        }  
    }  
    stage('Build & Run Tests') {  
        steps {  
            bat 'mvn clean test'  
        }  
        post {  
            always {  
                junit 'target/surefire-reports/*.xml'  
            }  
        }  
    }  
    stage('Package Application') {  
        steps {  
            bat 'mvn package -DskipTests'
```

```

    }
}
stage('Build Docker Image') {
    steps {
        bat "docker build -t %DOCKER_IMAGE% ."
    }
}
stage('Push to DockerHub') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'dockerhub-creds',
            usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_PASS'
        )]) {
            bat """
                docker login -u %DOCKER_USER% -p
%DOCKER_PASS%
                docker push %DOCKER_IMAGE%
            """
        }
    }
}
stage('Deploy to Kubernetes') {
    steps {

```

```
        bat 'kubectl apply -f deployment.yaml --validate=false --  
kubeconfig=C:\\Users\\deepa\\.kube\\config'
```

```
    }
```

```
  }
```

```
}
```

```
}
```

